

Energy Monitor Project:

This project is coded for an ATmega328P chip. The board measures the signal through the load that we connect through the two large pins at the top of our board. Circuit wise, this signal gets passed through the Op-Amp circuit to be amplified and passed to the ADC. One Op-Amp was connected to a current sensing circuit, and the other to a voltage sensing circuit, and level shifted.

Because of the nature of the ADC, each sample was taken slightly out of phase with each other. To counteract this and accurately calculate power we would store each reading and shift them so that their AC values are aligned.

adc.c controls the ADC setup and control. It contains the interrupts used to capture the values that are triggered by the comparator, a zero-crossing circuit. I believe we used the in-built one in the ATmega.

power.c performs the bulk of the calculations. It takes the samples and processes them into the final value that gets periodically sent to the display. First we remove the DC offset, which was included so the signal remains completely positive, and then we linearly interpolate it. Using this data we can get the measured power (P_{measured}) as well as the RMS values of voltage and current.

uart.c controls the communication via uart. This is used so we can connect the ISP header to our computer and read what it is doing, easily being able to add an extra `uart_transmit()` to `main.c` to see more information.

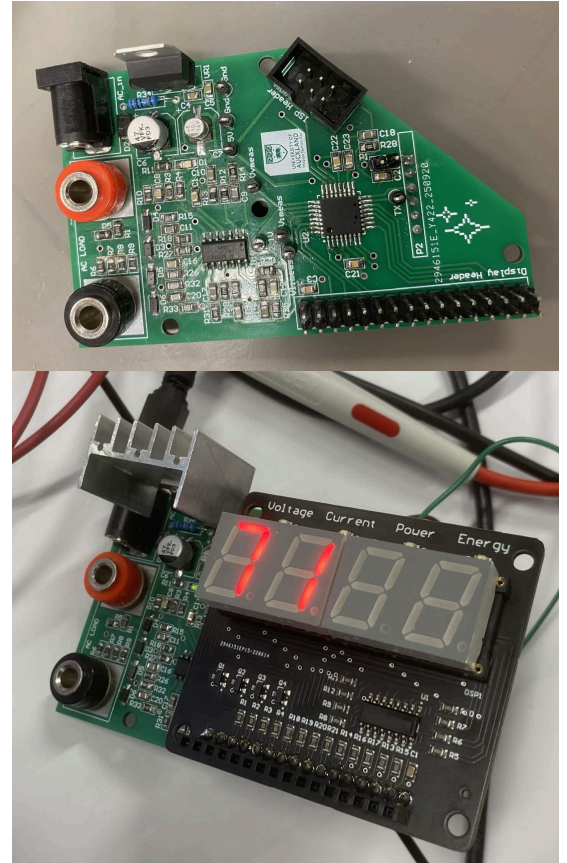
display.c controls the seven segment display array. We periodically feed it the calculated values, and it cycles through power, voltage, and current, so that uart is not needed to see its results and can instead be seen physically on the board.

We wrote our own simple math operations so we can avoid importing `Math.h`, as that api is massive and would bloat the storage of the chip. I also included our LTSpice files, which were used to simulate our Op-Amp circuit that eventually made it onto the board.

VHDL Flappy Bird:

In this Project we used a Cyclone V to program our version of flappy bird from scratch using VHDL. We had to be mindful of how our code would be synthesized, keeping track of latching and inefficiencies.

The various ROMs in the main folder are used to read the sprite data for each component of the game. These files are autogenerated by the compiler.



head.vhd is the top entity of the project. Here we define all of the components and interconnect them to create a functioning game. We also define any constants used elsewhere in this file.

fsm.vhd (located in the 'interfaces' folder) contains the finite state machine for the whole game. It controls what mode we're in, the lives we have left, and whether the game is paused, running, or other states. It takes orders from the UI. The main process keeps track of the lives, score, and difficulty. It also moves onto the next state when the state has changed. The other process is the state machine, which takes signals from the UI and queues up the next state if the conditions are correct.

layer_control.vhd (located in the 'interfaces' folder) takes in every colour channel supplied by each component of the system. It has a priority for each pixel from each channel, so that if two systems want to draw a pixel in the same location, it picks which one is highest on the list and sends only that one to the VGA display.

Ui.vhd controls the functionality of the UI. The first process reads what state we're in, and depending on what menu it draws boxes for buttons, and displays text inside them. The text was the first thing we got working in this project as it was very complicated. We read from the text ROM, and using a tiny bit of maths we can draw them in a line in any size we wanted (8x8, 16x16, 32x32, etc). The second process reads the mouse co-ordinates, and if the mouse is clicked inside a specific range of co-ordinates it sends out a button press signal to be used by the FSM.

bird.vhd controls the functionality of the flappy bird, which in our project is a jelly fish. It contains the rendering of the sprite, as well as its gravity and control functionality. It also has collision detection from the pipes and bonus stars. As the difficulty increases, a vignette closes around the bird, obscuring the view of the front of the screen, which is also controlled in this file. To detect a collision, we simply check if a bird pixel and a pipe pixel wants to draw at the same time, and if they do, a collision occurs. This way we don't need a hitbox.

background.vhd and floor.vhd do similar functions. They only render the background and a small strip at the bottom in front of the pipes onto the screen for decoration.

pipe.vhd controls the movement of the pipe, as well as if they have a bonus star between them. This component is duplicated 3 times, so that 3 pipes can be on the screen at once. Each pipe moves across the screen with the gap at a random height, and when they reach the end of the screen they teleport back to the start at a new height.

Java

I included these simple projects so you can see that I am proficient in object oriented programming.

Countries is a project about Maps, using LinkedLists to represent a graph. The countries are given to us in a csv file, as well as the connections between them, with the cost of travel. We implemented commands to be able to display the information of a country and its connections, as well as a 'ROUTE' command. This would allow us to pick two countries, and

using Breadth First Search returns the shortest route, as well as display information about the cost of travel.

Mind Games is a small colour guessing arcade game where you compete against an AI. You pick two colours; Your own colour and what you think your opponent will pick. You can select how hard you want the game which determines the AI's behaviour. We implemented 3 different strategies the AI can use. Random picks a random colour, LeastUsed picks a colour it hasn't used the most, and AvoidLast which picks a random colour except what the player used last. At the highest difficulty the AI adapts it's strategy, changing tactics depending on what's getting it the most points. This project used slightly more advanced methods such as builders, interfaces, and overrides.